МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОЛАТОРНОЇ №1

на тему **“ Основи функціонального програмування у Python ”**

Виконала:

Студентка 2 курсу

групи ФЕП-22

Сулятицька Христина

Перевірив:

Аспірант

Остріков Данило Андрійович

Львів-2026

Мета. ознайомитися з поняттями чистої функції та референтної прозорості; навчитися виявляти та ізолювати побічні ефекти; реалізувати обробку даних у функціональному стилі; навчитися використовувати typing.Callable для параметризації обчислень; не змінювати вхідні дані та створювати нові об'єкти під час обробки.

Теоретичні відомості.

Чиста функція — це функція, яка для однакових вхідних даних завжди повертає однаковий результат і не змінює зовнішній або глобальний стан програми.

Референтна прозорість означає, що результат виконання виразу можна замінити його значенням без зміни поведінки програми.

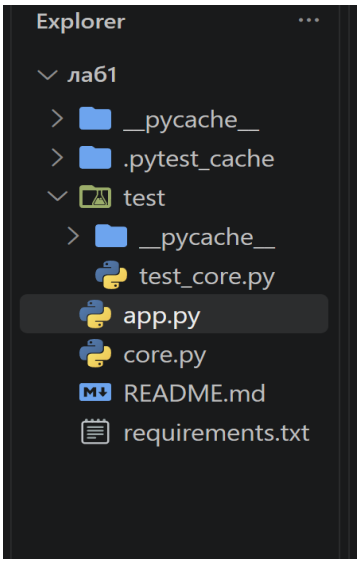
Побічні ефекти — це дії функції, які впливають на зовнішнє середовище: виведення через print, робота з файлами, мережею, зміна глобальних змінних або вхідних структур даних.

У функціональному стилі обчислювальну частину програми доцільно відокремлювати від введення та виведення даних. Вхідні об'єкти не змінюються, а замість них створюються нові об'єкти. Lab_01

Callable використовується для передавання функцій як аргументів. Це дозволяє окремо задавати правила фільтрації, конвертації та комісії й змінювати їх без зміни основного алгоритму обробки. Для 8-го варіанта через Callable передаються курс, комісія та критерій мінімальної суми транзакції.

Хід роботи

1. **Створено структуру проєкту** лабораторної роботи:



2. Реалізовано чисте ядро програми у файлі core.py. У ньому створено функції для:

- перевірки успішності транзакції;
- конвертації суми;
- застосування комісії;
- створення нового об'єкта обробленої транзакції;
- формування налаштованого обробника транзакцій.



```python
from typing import Callable, TypedDict


class Transaction(TypedDict):
    id: int
    type: str
    amount: float
    currency: str
    status: str


class ProcessedTransaction(TypedDict):
    id: int
    type: str
    amount: float
    currency: str
    status: str
    converted_amount: float
    final_amount: float


RateFn = Callable[[str], float]
CommissionFn = Callable[[float], float]
FilterFn = Callable[[float], bool]

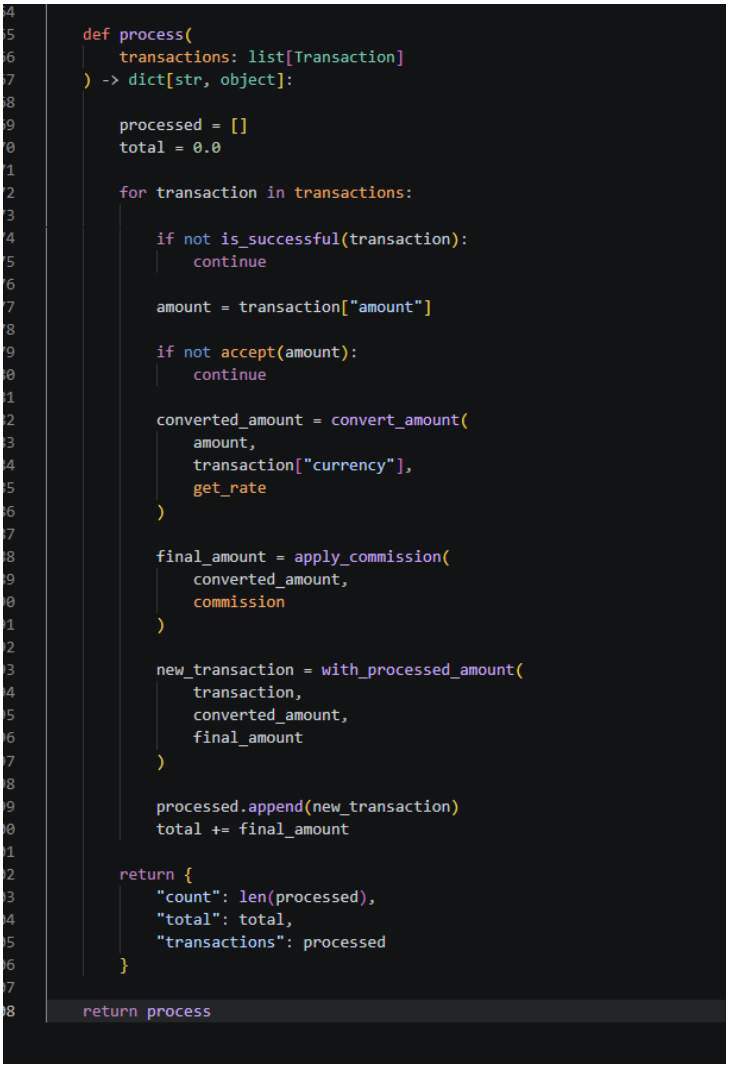

def is_successful(transaction: Transaction) -> bool:
    return transaction["status"] == "successful"


def convert_amount(
    amount: float,
    currency: str,
    get_rate: RateFn
) -> float:
    return amount * get_rate(currency)


def apply_commission(
    amount: float,
    commission: CommissionFn
) -> float:
    return commission(amount)


def with_processed_amount(
    transaction: Transaction,
    converted_amount: float,
    final_amount: float
) -> ProcessedTransaction:
    return {
        **transaction,
        "converted_amount": converted_amount,
        "final_amount": final_amount
    }


def make_processor(
    *,
    get_rate: RateFn,
    commission: CommissionFn,
    accept: FilterFn
) -> Callable[[list[Transaction]], dict[str, object]]:
```



```python
    def process(
        transactions: list[Transaction]
    ) -> dict[str, object]:

        processed = []
        total = 0.0

        for transaction in transactions:

            if not is_successful(transaction):
                continue

            amount = transaction["amount"]

            if not accept(amount):
                continue

            converted_amount = convert_amount(
                amount,
                transaction["currency"],
                get_rate
            )

            final_amount = apply_commission(
                converted_amount,
                commission
            )

            new_transaction = with_processed_amount(
                transaction,
                converted_amount,
                final_amount
            )

            processed.append(new_transaction)
            total += final_amount

        return {
            "count": len(processed),
            "total": total,
            "transactions": processed
        }

    return process
```

Реалізовано обробку банківських транзакцій. Програма спочатку відбирає лише транзакції зі статусом successful, після чого перевіряє мінімальну суму. Транзакції, які не відповідають заданим умовам, не обробляються.**Виконано конвертацію валют.** Для кожної прийнятної транзакції сума переводиться у гривні відповідно до переданого курсу. **Застосовано комісію.** Після конвертації до отриманої суми застосовується функція комісії. **Забезпечено відсутність мутації вхідних даних.** Замість зміни початкового словника транзакції створюється новий словник із додатковими полями converted_amount та final_amount. **Використано Callable.** Курс валюти, функція комісії та критерій прийнятності передаються в make_processor як функції. Це дає можливість змінювати правила обробки без зміни основного коду.

3. **Побічні ефекти винесено в app.py.** Виведення результатів за допомогою print виконується лише в оболонці програми, а не в чистих функціях. Такий поділ відповідає вимозі ізоляції I/O.



```python
from core import make_processor


def get_rate(currency: str) -> float:
    rates = {
        "USD": 41.0,
        "EUR": 48.0,
        "UAH": 1.0
    }

    return rates.get(currency, 1.0)


def commission(amount: float) -> float:
    return amount * 0.98


def accept(amount: float) -> bool:
    return amount >= 50


processor = make_processor(
    get_rate=get_rate,
    commission=commission,
    accept=accept
)


transactions = [
    {
        "id": 1,
        "type": "payment",
        "amount": 100,
        "currency": "USD",
        "status": "successful"
    },
    {
        "id": 2,
        "type": "transfer",
        "amount": 30,
        "currency": "EUR",
        "status": "successful"
    },
    {
        "id": 3,
        "type": "payment",
        "amount": 200,
        "currency": "USD",
        "status": "failed"
    },
    {
        "id": 4,
        "type": "payment",
        "amount": 150,
        "currency": "EUR",
        "status": "successful"
    }
]


result = processor(transactions)


for transaction in result["transactions"]:
    print(
        "Transaction:",
        transaction["id"],
        "final amount:",
        transaction["final_amount"]
    )

print("Count:", result["count"])
print("Total:", result["total"])
```

4. Створено тести pytest для перевірки:

- референтної прозорості;
- відсутності зміни вхідних даних;
- правильності роботи переданих Callable-функцій.

Виконано тестування програми. У результаті виконання команди python -m pytest



```python
from copy import deepcopy

from core import make_processor


def test_referential_transparency():
    transactions = [
        {
            "id": 1,
            "type": "payment",
            "amount": 100,
            "currency": "USD",
            "status": "successful"
        }
    ]

    processor = make_processor(
        get_rate=lambda currency: 40.0,
        commission=lambda amount: amount * 0.9,
        accept=lambda amount: amount >= 50
    )

    result1 = processor(transactions)
    result2 = processor(transactions)

    assert result1 == result2


def test_no_mutation():
    transactions = [
        {
            "id": 1,
            "type": "payment",
            "amount": 100,
            "currency": "USD",
            "status": "successful"
        }
    ]

    original = deepcopy(transactions)

    processor = make_processor(
        get_rate=lambda currency: 40.0,
        commission=lambda amount: amount * 0.9,
        accept=lambda amount: amount >= 50
    )

    processor(transactions)

    assert transactions == original

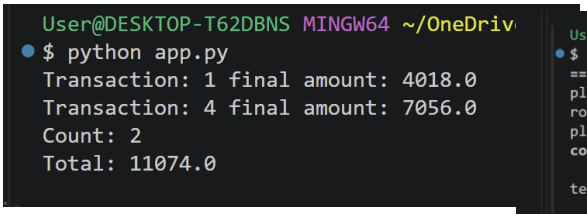

def test_callable_policies():
    transactions = [
        {
            "id": 1,
            "type": "payment",
            "amount": 100,
            "currency": "USD",
            "status": "successful"
        }
    ]

    processor = make_processor(
        get_rate=lambda currency: 40.0,
        commission=lambda amount: amount * 0.9,
        accept=lambda amount: amount >= 50
    )

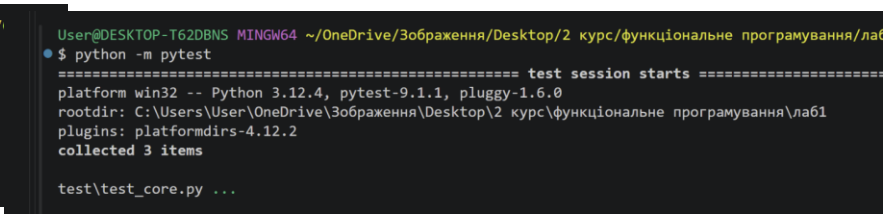
    result = processor(transactions)

    assert result["count"] == 1
    assert result["total"] == 3600.0
```

Результат:



```
User@DESKTOP-T62DBNS MINGW64 ~/OneDriv
$ python app.py
Transaction: 1 final amount: 4018.0
Transaction: 4 final amount: 7056.0
Count: 2
Total: 11074.0
```



```
User@DESKTOP-T62DBNS MINGW64 ~/OneDrive/Зображення/Desktop/2 курс/функціональне програмування/ла
$ python -m pytest
============================================= test session starts ==========
platform win32 -- Python 3.12.4, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\User\OneDrive\Зображення\Desktop\2 курс\функціональне програмування\ла61
plugins: platformdirs-4.12.2
collected 3 items

test\test_core.py ...
```

Висновок: У ході лабораторної роботи було реалізовано програму для аналізу банківських транзакцій відповідно до 8-го варіанта. Було застосовано принципи функціонального програмування: чисті функції, відсутність мутації вхідних даних, ізоляцію побічних ефектів та явну передачу параметрів. Для параметризації правил обробки використано typing.Callable, а для створення налаштованого обробника — функцію вищого порядку make_processor. Роботу програми перевірено за допомогою pytest: усі 3 тести пройшли успішно. Обов'язкові вимоги лабораторної щодо чистих функцій, Callable, відсутності мутацій, ізоляції I/O та тестування були виконані.